Fundação Escola de Comércio Álvares Penteado - FECAP
CIÊNCIA DA COMPUTAÇÃO - 6º SEMESTRE

Caroliny Rossi Bittencourt – 24025959
Duda Lucena Miguel – 24025889
Gabriel Debastiani Davanço – 24025899
Ricardo Liyudi Tetsuya – 24026553

ENTREGA 2 - SISTEMAS EMBARCADOS E ROBÓTICA

RELATÓRIO TÉCNICO DE PROVA DE CONCEITO (PoC)

Dispositivo Wearable para Reconhecimento de Movimentos no Tênis via IMU e IA

São Paulo – SP
2026

1 INTRODUÇÃO E DESCRIÇÃO DO DISPOSITIVO

O projeto consiste no desenvolvimento e na validação da Prova de Conceito (*Proof of Concept* – PoC) de um dispositivo eletrônico vestível (*wearable*) voltado à telemetria esportiva e análise biomecânica aplicada ao tênis. A finalidade primária do dispositivo é a aquisição contínua de variáveis cinemáticas para a posterior classificação supervisionada de golpes (e.g., *forehand*, *backhand*, saque) através de algoritmos de inteligência artificial.

Para a mensuração inercial, o sistema emprega a Unidade de Medição Inercial (*Inertial Measurement Unit* – IMU) MPU-6050, que opera com 6 Graus de Liberdade (6-DoF), fornecendo leituras tridimensionais de aceleração linear (a_x, a_y, a_z) e velocidade angular ($\omega_x, \omega_y, \omega_z$). Os dados são processados e integrados a uma cadeia computacional que engloba rotulagem de séries temporais, treinamento de modelos e inferência de padrões de movimento.

2 ARQUITETURA E FLUXO DE FUNCIONAMENTO

O ciclo de vida operacional do dispositivo organiza-se em duas etapas interdependentes:

2.1 Etapa 1: Aquisição de Dados e Modelagem de Inteligência Artificial (Offline)

A primeira etapa corresponde ao processo de extração e tratamento dos dados para treinamento do classificador estatístico/preditivo:

1. O microcontrolador lê os registradores do sensor MPU-6050 via barramento I^2C e transmite os vetores brutos pela interface Serial (*Universal Asynchronous Receiver-Transmitter* – UART/USB) em taxa amostral periódica;

2. Um script em linguagem Python captura o fluxo de dados no computador hospedeiro via biblioteca de porta serial, efetuando o desacoplamento das variáveis nos seis eixos cinemáticos;

3. Os dados são consolidados juntamente a uma coluna de índice temporal (*timestamp* / *frame*), gerando conjuntos de dados estruturados em formato `.csv` devidamente rotulados por classe de golpe para alimentação do pipeline de treinamento de modelos de aprendizado de máquina.

2.2 Etapa 2: Operação no Ecossistema Integrado e Inferência (Online)

A segunda etapa estabelece o funcionamento autônomo do dispositivo integrado ao ecossistema de aplicação do usuário:

1. O microcontrolador coleta as leituras inerciais e realiza o empacotamento em formato hexadecimal para redução de *overhead* de transmissão;

2. Os pacotes são transmitidos sem fio por meio do protocolo *Bluetooth Low Energy* (BLE) para o cliente operacional (aplicativo móvel ou cliente *web* via *Web Bluetooth API*);

3. A aplicação receptora executa a decodificação dos bytes hexadecimais, o pré-processamento de filtragem de sinal e submete os tensores de entrada ao modelo de inferência hospedado na API *backend*, retornando em tempo real a classificação do movimento executado pelo atleta.

3 MONTAGEM E PROVA DE CONCEITO DO PROTÓTIPO

A montagem física implementada corresponde a um protótipo laboratorial de prova de conceito, voltado estritamente à comprovação da viabilidade funcional de aquisição e integridade da transmissão de dados no ecossistema proposto:

- **Plataforma de Processamento:** Utiliza-se a placa Arduino UNO R4 WiFi, responsável pela gestão dos protocolos de comunicação, temporização e leitura dos periféricos;
- **Sensoriamento:** O sensor MPU-6050 é acoplado diretamente aos barramentos de comunicação I^2C (pinos SDA e SCL) e referências de tensão;
- **Alimentação Autônoma:** Para viabilizar a mobilidade sem cabos conectados à rede elétrica, o circuito é alimentado por uma associação de duas baterias de polímero de lítio (LiPo) de 3,7 V em série (totalizando 7,4 V nominais), intermediadas por módulo de gerenciamento de carga;
- **Aspectos Construtivos:** Nesta iteração de prova de conceito, o protótipo não conta com carcaça ou encapsulamento selado, operando com cabeamento direto e suporte em protoboard para viabilizar medições elétricas imediatas, inspeção de hardware e validação dos barramentos lógicos.

4 TECNOLOGIAS E FERRAMENTAS EMPREGADAS

A Tabela 1 sintetiza a especificação e o papel desempenhado por cada tecnologia de hardware e software dentro da arquitetura do projeto.

Tabela 1 – Especificação das Tecnologias e Ferramentas do Projeto

Camada / Domínio	**Componente / Tecnologia**	**Função no Sistema**
Hardware Embarcado	Arduino UNO R4 WiFi	Microcontrolador mestre, execução do firmware, processamento I^2C e transmissão BLE.
	MPU-6050	Sensor inercial de 6 graus de liberdade (aceleração linear e velocidade angular X, Y, Z).
	2x Baterias LiPo 3,7 V (Série)	Fonte de alimentação autônoma de 7,4 V nominais para operação desvinculada da rede.
	Módulo de Carregamento	Circuito integrado de proteção e recarga segura das células de lítio.
Edge / Firmware	Arduino IDE / C++	Desenvolvimento do código embarcado, leitura por interrupções e empacotamento de dados.
Frontend	React Native (TypeScript)	Aplicação cliente para pareamento Bluetooth, recepção do fluxo e interface do usuário.
Backend e Dados	FastAPI (Python)	Servidor REST assíncrono para ingestão de séries temporais e orquestração do modelo de IA.
	SQLite / TinyDB	Persistência local e estruturação dos registros de sessões, métricas e histórico de treinos.

5 DEMONSTRAÇÃO PRÁTICA DO PROTÓTIPO

Para demonstrar o funcionamento e a validação em bancada da Prova de Conceito (PoC) desenvolvida — evidenciando a alimentação autônoma por baterias LiPo, a sinalização visual via matriz de LEDs 12x8 nativa do Arduino UNO R4 WiFi e o display OLED, bem como a comunicação em tempo real —, disponibilizou-se o registro audiovisual do protótipo em operação:

- **Vídeo de Demonstração (YouTube):** https://youtube.com/shorts/tBKtHX7DJaE

6 CÓDIGO-FONTE DO SISTEMA EMBARCADO

O Código 1 apresenta a implementação completa do firmware embarcado desenvolvido em C++ para a placa Arduino UNO R4 WiFi. A lógica contempla a ini-

cialização do barramento I^2C, integração da matriz de matriz de LEDs 12x8 nativa, suporte ao display OLED SSD1306 e configuração dos serviços GATT para comunicação *Bluetooth Low Energy* (BLE).

Listing 1: Firmware do Wearable FLUX (Arduino UNO R4 WiFi)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ArduinoBLE.h>
#include "Arduino_LED_Matrix.h"

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1

//
    ================================================================================

// METADADOS DO SISTEMA EMBARCADO (ENVIADOS APENAS 1x NO HANDSHAKE BLE)
//
    ================================================================================

#define DEVICE_NAME     "FLUX-Tennis-R4"
#define BOARD_MODEL     "Arduino UNO R4 WiFi"
#define FIRMWARE_VER    "v1.0.0"

// Instancia a Matriz de LEDs 12x8 nativa do Arduino UNO R4 WiFi
ArduinoLEDMatrix matrix;

// Bitmap 8x12:  cone  "X" (Desconectado / Aguardando Bluetooth)
uint8_t ICON_DESCONECTADO_X[8][12] = {
  { 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0 },
  { 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0 },
  { 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0 },
  { 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0 },
  { 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0 }
};

// Bitmap 8x12:  cone  "Check  " (Conectado via Bluetooth ao App)
uint8_t ICON_CONECTADO_CHECK[8][12] = {
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1 },
```

```
  { 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0 },
  { 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0 },
  { 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0 },
  { 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0 },
  { 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0 }
};

// Instancia o display OLED no barramento I2C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
bool oledDisponivel = false;

// Instancia o sensor de movimento IMU MPU6050 (Acelerômetro + Giroscópio)
Adafruit_MPU6050 mpu;
bool mpuDisponivel = false;

// ============================================================================

// CONFIGURAÇÃO BLUETOOTH LOW ENERGY (BLE) - COMPATÍVEL COM O FRONTEND FLUX
// ============================================================================

// Serviço padrão de Bateria BLE (0x180F)
BLEService batteryService("180F");
BLEUnsignedCharCharacteristic batteryLevelChar("2A19", BLERead | BLENotify);

// Serviço principal do FLUX Smart Tennis
BLEService fluxImuService("4fafc201-1fb5-459e-8fcc-c5c9c331914b");

// Característica 1: Ficha Estática do Aparelho (Modelo|Versão) -> Lida apenas 1x na conexão!
BLEStringCharacteristic fluxDeviceInfoChar(
  "beb5483e-36e1-4688-b7f5-ea07361b26a9",
  BLERead,
  64
);

// Característica 2: Stream de Telemetria IMU durante o treino
BLEStringCharacteristic fluxTelemetryChar(
  "beb5483e-36e1-4688-b7f5-ea07361b26a8",
  BLERead | BLENotify | BLEWrite,
  64
```

```
);

// Intervalo de transmissão (50ms = 20 Hz, ideal para calcular delta t e dinâmicas de movimento)
#define SAMPLE_INTERVAL_MS 50

unsigned long ultimoEnvioMs = 0;
unsigned long ultimoDisplayMs = 0;
unsigned long contadorFrames = 0;
uint8_t nivelBateria = 96;
bool estavaConectado = false;

// Atualiza a Matriz de LEDs 12x8 da placa UNO R4 WiFi (Check = conectado, X = desconectado)
void atualizarMatrizLED(bool conectado) {
  if (conectado) {
    matrix.renderBitmap(ICON_CONECTADO_CHECK, 8, 12);
  } else {
    matrix.renderBitmap(ICON_DESCONECTADO_X, 8, 12);
  }
}

// Endereço I2C detectado para o sensor MPU6050 (0x68 ou 0x69)
uint8_t mpuEnderecoI2C = 0x68;
int leiturasZeradasConsecutivas = 0;

// Re-acorda o MPU6050 caso ruídos ou timeout no I2C o coloquem em Sleep Mode (PWR_MGMT_1)
void acordarMPU() {
  Wire.beginTransmission(mpuEnderecoI2C);
  Wire.write(0x6B); // Registrador PWR_MGMT_1
  Wire.write(0x00); // Zera o bit de Sleep
  Wire.endTransmission(true);
}

// Monta o payload padronizado com Frame, Timestamp, 6 eixos, Temperatura e Bateria:
// Formato: "IMU:FRAME,MILLIS,AX,AY,AZ,GX,GY,GZ,TEMP,BAT"
String obterPayloadIMU(unsigned long frameId, unsigned long timestampMs) {
  String payload = "IMU:" + String(frameId) + "," + String(timestampMs) + ",";

  if (mpuDisponivel) {
    sensors_event_t a, g, temp;
    if (mpu.getEvent(&a, &g, &temp)) {
      // Se todos os 3 eixos de aceleração vierem exatamente 0.00, o
```

```
    sensor entrou em Sleep ou travou
118       if (a.acceleration.x == 0.0f && a.acceleration.y == 0.0f && a.
    acceleration.z == 0.0f) {
119         leiturasZeradasConsecutivas++;
120         if (leiturasZeradasConsecutivas > 3) {
121           acordarMPU(); // Força acordar o sensor
122           leiturasZeradasConsecutivas = 0;
123         }
124       } else {
125         leiturasZeradasConsecutivas = 0;
126       }
127 
128       payload += String(a.acceleration.x, 2) + ",";
129       payload += String(a.acceleration.y, 2) + ",";
130       payload += String(a.acceleration.z, 2) + ",";
131       payload += String(g.gyro.x, 2) + ",";
132       payload += String(g.gyro.y, 2) + ",";
133       payload += String(g.gyro.z, 2) + ",";
134       payload += String(temp.temperature, 1) + ",";
135       payload += String(nivelBateria);
136     } else {
137       acordarMPU();
138       payload += "0.00,0.00,0.00,0.00,0.00,0.00,0.0," + String(
    nivelBateria);
139     }
140   } else {
141     // Fallback caso o sensor I2C nao responda
142     payload += "0.00,0.00,0.00,0.00,0.00,0.00,0.0," + String(
    nivelBateria);
143   }
144 
145   return payload;
146 }
147 
148 // Atualiza o display OLED com o status atual do Bluetooth e telemetria
149 void atualizarDisplay(bool conectado, String enderecoCentral) {
150   if (!oledDisponivel) return;
151 
152   display.clearDisplay();
153   display.setTextSize(1);
154   display.setTextColor(SSD1306_WHITE);
155 
156   // Cabeçalho
157   display.setCursor(0, 0);
158   display.print(DEVICE_NAME);
159   display.print(" ");
160   display.println(FIRMWARE_VER);
```

```
161   display.drawLine(0, 11, 127, 11, SSD1306_WHITE);
162
163   // Status da conexão Bluetooth
164   display.setCursor(0, 16);
165   if (conectado) {
166     display.println("BLE: CONECTADO! [OK]");
167     display.setCursor(0, 28);
168     display.print("App: ");
169     display.println(enderecoCentral);
170   } else {
171     display.println("BLE: DESCONECTADO [X]");
172     display.setCursor(0, 28);
173     display.println(BOARD_MODEL);
174   }
175
176   // Rodapé com contador de frames de telemetria enviados
177   display.setCursor(0, 44);
178   display.print("Frames IMU: ");
179   display.println(contadorFrames);
180
181   display.setCursor(0, 55);
182   display.print("Bateria: ");
183   display.print(nivelBateria);
184   display.println("%");
185
186   display.display();
187 }
188
189 void setup() {
190   Serial.begin(115200);
191   delay(1000);
192
193   // 1. Inicializa a Matriz de LEDs 12x8 do UNO R4 WiFi (começa exibindo
        "X")
194   matrix.begin();
195   atualizarMatrizLED(false);
196
197   // 2. Inicializa barramento I2C em Fast Mode (400 kHz) para evitar
        colisão entre OLED e MPU6050
198   Wire.begin();
199   Wire.setClock(400000); // 400 kHz Fast Mode (libera o barramento
        rapidamente)
200   Wire.setTimeout(50);
201
202   Serial.println("\n--- INICIALIZANDO FLUX WEARABLE (UNO R4 WIFI) ---");
203
204   if (display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
```

```
205     oledDisponivel = true;
206   } else if (display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) {
207     oledDisponivel = true;
208   } else {
209     Serial.println("[AVISO] Display OLED nao detectado. Usando Matriz de
      LEDs 12x8 + BLE.");
210   }
211
212   // 3. Inicializa o sensor MPU6050 (Acelerometro + Giroscópio)
213   if (mpu.begin(0x68)) {
214     mpuDisponivel = true;
215     mpuEnderecoI2C = 0x68;
216     mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
217     mpu.setGyroRange(MPU6050_RANGE_500_DEG);
218     mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
219     acordarMPU();
220     Serial.println("[OK] Sensor MPU6050 inicializado no endereco 0x68!")
      ;
221   } else if (mpu.begin(0x69)) {
222     mpuDisponivel = true;
223     mpuEnderecoI2C = 0x69;
224     mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
225     mpu.setGyroRange(MPU6050_RANGE_500_DEG);
226     mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
227     acordarMPU();
228     Serial.println("[OK] Sensor MPU6050 inicializado no endereco 0x69!")
      ;
229   } else {
230     Serial.println("[AVISO] Sensor MPU6050 nao encontrado no barramento
      I2C (verifique pinos SDA/SCL, alimentacao e AD0 no GND).");
231   }
232
233   // 4. Inicializa o módulo Bluetooth Low Energy (ArduinoBLE)
234   if (!BLE.begin()) {
235     Serial.println("[ERRO] Falha ao iniciar o modulo Bluetooth Low
      Energy (BLE)!");
236     while (1) {
237       delay(1000);
238     }
239   }
240
241   // Define o nome visível no navegador (Web Bluetooth) e no Smartphone
242   BLE.setDeviceName(DEVICE_NAME);
243   BLE.setLocalName(DEVICE_NAME);
244
245   // Configura e adiciona o serviço de Bateria (0x180F)
246   batteryService.addCharacteristic(batteryLevelChar);
```

```
247   BLE.addService(batteryService);
248   batteryLevelChar.writeValue(nivelBateria);
249
250   // Configura e adiciona o serviço principal do FLUX
251   fluxImuService.addCharacteristic(fluxDeviceInfoChar);
252   fluxImuService.addCharacteristic(fluxTelemetryChar);
253   BLE.addService(fluxImuService);
254
255   // Registra o Modelo da Placa e a Versão do Software UMA  NICA  VEZ no
        registrador GATT
256   // Formato: "Modelo da Placa|Versao do Software" (lido pelo Frontend
       apenas no handshake)
257   String handshakeMetadata = String(BOARD_MODEL) + "|" + String(
       FIRMWARE_VER);
258   fluxDeviceInfoChar.writeValue(handshakeMetadata);
259   fluxTelemetryChar.writeValue("FLUX_READY");
260
261   // Anuncia o serviço principal para ser encontrado no scan
262   BLE.setAdvertisedService(fluxImuService);
263
264   // Inicia o advertising BLE
265   BLE.advertise();
266
267   Serial.print("[OK] BLE Ativo! Metadados embarcados registrados: ");
268   Serial.println(handshakeMetadata);
269   atualizarDisplay(false, "");
270 }
271
272 void loop() {
273   // Escuta conexões de um dispositivo Central (Navegador Web ou
       Smartphone)
274   BLEDevice central = BLE.central();
275
276   if (central) {
277     if (!estavaConectado) {
278       estavaConectado = true;
279       Serial.print("[BLE] Conectado ao App! MAC Central: ");
280       Serial.println(central.address());
281
282       // Garante que o nível de bateria inicial está atualizado no
      handshake
283       batteryLevelChar.writeValue(nivelBateria);
284
285       // Desenha o CHECK ( ) na Matriz de LEDs 12x8 do Arduino UNO R4
      WiFi!
286       atualizarMatrizLED(true);
287       atualizarDisplay(true, central.address());
```

```
288     }
289
290     // Enquanto o app estiver conectado ao Arduino
291     while (central.connected()) {
292       unsigned long agora = millis();
293
294       // Envia telemetria IMU em alta frequencia via BLE e Serial
295       if (agora - ultimoEnvioMs >= SAMPLE_INTERVAL_MS) {
296         ultimoEnvioMs = agora;
297         contadorFrames++;
298
299         String payload = obterPayloadIMU(contadorFrames, agora);
300
301         // 1. Envia via Bluetooth BLE
302         fluxTelemetryChar.writeValue(payload);
303
304         // 2. Envia exatamente o mesmo pacote via Serial (115200 baud)
305         Serial.println(payload);
306       }
307
308       // Atualiza o display OLED em taxa menor (1 segundo) para nao
    concorrer no barramento I2C com o MPU
309       if (agora - ultimoDisplayMs >= 1000) {
310         ultimoDisplayMs = agora;
311         atualizarDisplay(true, central.address());
312       }
313
314       BLE.poll();
315     }
316
317     // Quando o app desconectar, volta a desenhar o X na Matriz de LEDs
    12x8!
318     estavaConectado = false;
319     Serial.print("[BLE] Desconectado de: ");
320     Serial.println(central.address());
321     atualizarMatrizLED(false);
322     atualizarDisplay(false, "");
323   } else {
324     unsigned long agora = millis();
325     if (agora - ultimoEnvioMs >= SAMPLE_INTERVAL_MS) {
326       ultimoEnvioMs = agora;
327       contadorFrames++;
328
329       String payload = obterPayloadIMU(contadorFrames, agora);
330       Serial.println(payload);
331     }
332
```

```
333     if (agora - ultimoDisplayMs >= 1000) {
334       ultimoDisplayMs = agora;
335       atualizarDisplay(false, "");
336     }
337   }
338 }
```